PyTorch Autograd: A Complete Beginner's Guide

Automatic Differentiation for Deep Learning

Deep Learning Fundamentals

March 23, 2026

Contents

1 Introduction: Why Do We Need Autograd?

Before understanding what Autograd is, we need to understand **why** it is considered one of the most important modules in PyTorch. The motivation comes from a fundamental challenge in deep learning: computing derivatives efficiently.

1.1 The Manual Derivative Problem

Consider a simple mathematical relationship:

$$y = x^2 \tag{1}$$

Challenge: Write a Python program that can compute the derivative $\frac{dy}{dx}$ for any given value of x.

```python
def dy_dx(x):
    return 2 * x  # Since d/dx(x^2) = 2x

# Testing
print(dy_dx(2))  # Output: 4
print(dy_dx(3))  # Output: 6
```

Listing 1: Manual Derivative for Simple Function

This is straightforward. But what if the function becomes more complex?

1.2 Nested Functions and the Chain Rule

Now consider a nested function scenario:

$$y = x^2, \quad z = \sin(y) \tag{2}$$

New Challenge: Compute $\frac{dz}{dx}$ for any given x.

Mathematical Derivation (Chain Rule) To find $\frac{dz}{dx}$, we apply the chain rule:

$$\frac{dz}{dx} = \frac{dz}{dy} \times \frac{dy}{dx} \tag{3}$$

Where:

- $\frac{dz}{dy} = \cos(y) = \cos(x^2)$
- $\frac{dy}{dx} = 2x$

Therefore:

$$\frac{dz}{dx} = \cos(x^2) \times 2x = 2x\cos(x^2) \tag{4}$$

```python
import math

def dz_dx(x):
    return 2 * x * math.cos(x**2)

```

```
# Testing
print(dz_dx(3))  # Output: -5.4662...
print(dz_dx(4))  # Output: -7.6612...
```

Listing 2: Manual Derivative for Nested Function

Observation: The second example was more difficult than the first because it involved:

- Nested functions (composition of functions)
- Application of the chain rule
- Manual derivation and coding of the final formula

1.3 Adding More Complexity

What if we add another level of nesting?

$$y = x^2, \quad z = \sin(y), \quad w = e^z \tag{5}$$

Now to find $\frac{dw}{dx}$, we need:

$$\frac{dw}{dx} = \frac{dw}{dz} \times \frac{dz}{dy} \times \frac{dy}{dx} \tag{6}$$

Key Insight: As nested functions become deeper, manual derivative computation and coding becomes exponentially more difficult!

2 Connection to Deep Learning

You might wonder: *Why are we studying nested functions and their derivatives?* The answer is that **neural networks are essentially deeply nested functions**.

2.1 Example: Training a Simple Neural Network

Consider a real-world scenario: predicting whether a student will get placed based on their CGPA.

Dataset Structure:

- Input: CGPA (single feature)
- Output: Placement status (0 or 1)

Network Architecture: A single neuron with sigmoid activation (perceptron).

2.2 The Four Steps of Training

Step 1: Forward Pass: Send input through the network to get prediction

$$z = wx + b \tag{7}$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{8}$$

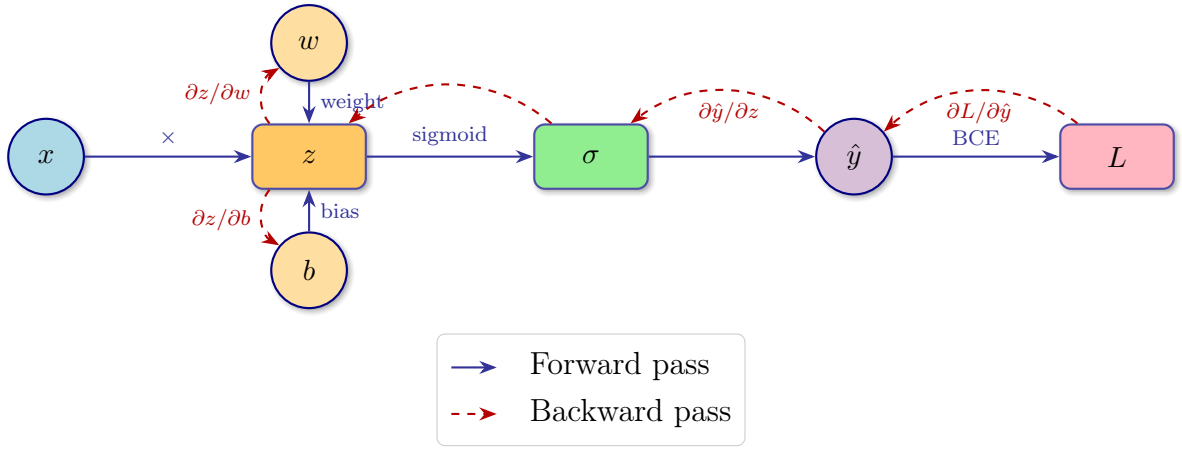

Figure 1: Single Neuron Neural Network with Forward Pass

Step 2: Loss Calculation: Compare prediction with actual value

$$L = \text{Binary Cross Entropy}(\hat{y}, y) \tag{9}$$

Step 3: Backward Pass (Backpropagation): Compute gradients of loss with respect to parameters

$$\frac{\partial L}{\partial w} = ? \tag{10}$$

$$\frac{\partial L}{\partial b} = ? \tag{11}$$

Step 4: Parameter Update: Adjust weights using gradient descent

$$w = w - \alpha \frac{\partial L}{\partial w} \tag{12}$$

$$b = b - \alpha \frac{\partial L}{\partial b} \tag{13}$$

2.3 The Nested Function Nature

Observe the mathematical equations:

- First, we use w and b to calculate z
- Then, we use z to calculate $\hat{y}$ (y_pred)
- Finally, we use $\hat{y}$ to calculate L

Critical Observation: The loss L is **indirectly** connected to w and b. This entire neural network is a **nested function**!

Manual Backpropagation Derivatives To compute $\frac{\partial L}{\partial w}$ manually, we need the chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w} \tag{14}$$

Similarly for bias:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial b} \tag{15}$$

Where:

- $\frac{\partial z}{\partial w} = x$ (input feature)
- $\frac{\partial z}{\partial b} = 1$ (bias derivative)
- $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$ (sigmoid derivative)
- $\frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}$ (BCE derivative)

Multiplying these together:

$$\frac{\partial L}{\partial w} = (\hat{y} - y) \times x \tag{16}$$

$$\frac{\partial L}{\partial b} = (\hat{y} - y) \times 1 \tag{17}$$

The Problem: In a complex neural network with thousands of weights and multiple layers, manually computing every derivative is **nearly impossible**!

3 What is Autograd?

Formal Definition: **Autograd** is a core component of PyTorch that provides **automatic differentiation**. It enables gradient computation, which is essential for training machine learning and deep learning models using optimization algorithms like gradient descent.
Autograd automatically computes gradients for all operations performed on tensors, eliminating the need for manual derivative calculations.

4 Understanding Autograd Through Examples

4.1 Example 1: Simple Function $y = x^2$

Let's see how Autograd handles our first simple example.

```python
import torch

# Create tensor with gradient tracking enabled
x = torch.tensor(3.0, requires_grad=True)

# Define the relationship
y = x ** 2

print(f"x = {x}")
print(f"y = {y}")
```

Listing 3: Autograd Implementation of $y = x^2$

Output:

```
x = tensor(3., requires_grad=True)
y = tensor(9., grad_fn=<PowBackward0>)
```

Key Observations

- `requires_grad=True`: Tells PyTorch we want to compute gradients for this tensor
- `grad_fn=<PowBackward0>`: PyTorch tracks that y was created by squaring x. This is a reference to the **backward** function that will compute the derivative.

4.1.1 The Computation Graph

When you set `requires_grad=True`, PyTorch internally creates a **computation graph**:

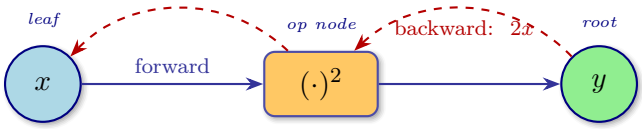

Figure 2: Computation Graph: Forward and Backward Pass

- **Forward Direction:** Calculate y from x (shown by black arrows)
- **Backward Direction:** Calculate $\frac{dy}{dx}$ (shown by red dashed arrows)

4.1.2 Computing the Gradient

```python
# Perform backward pass
y.backward()

# Access the computed gradient
print(f"dy/dx at x=3: {x.grad}")
```

Listing 4: Computing Derivative with Autograd

Output:

```
dy/dx at x=3: tensor(6.)
```

Verification: $\frac{d}{dx}(x^2) = 2x = 2 \times 3 = 6$

The Magic Formula The pattern for using Autograd is always:

1. Create input tensor with `requires_grad=True`
2. Perform operations (forward pass)
3. Call `.backward()` on the output
4. Access gradients using `.grad` on the input tensor

4.2 Example 2: Nested Function $z = \sin(x^2)$

Now let's tackle the more complex nested function.

```python
import torch
import math

# Manual calculation for verification
def dz_dx_manual(x):
    return 2 * x * math.cos(x**2)

print(f"Manual dz/dx at x=4: {dz_dx_manual(4)}")

# Autograd approach
x = torch.tensor(4.0, requires_grad=True)
y = x ** 2          # y = x^2
z = torch.sin(y)    # z = sin(y)

print(f"\nx = {x}")
print(f"y = {y}")
print(f"z = {z}")

# Backward pass
z.backward()

print(f"\nAutograd dz/dx at x=4: {x.grad}")
```

Listing 5: Autograd with Nested Functions

Output:

```
Manual dz/dx at x=4: -7.661275842587077

x = tensor(4., requires_grad=True)
y = tensor(16., grad_fn=<PowBackward0>)
z = tensor(-0.2879, grad_fn=<SinBackward0>)

Autograd dz/dx at x=4: tensor(-7.6613)
```

Perfect match! Autograd automatically applied the chain rule:

$$\frac{dz}{dx} = \frac{dz}{dy} \times \frac{dy}{dx} = \cos(y) \times 2x = \cos(x^2) \times 2x \tag{18}$$

4.2.1 The Full Computation Graph

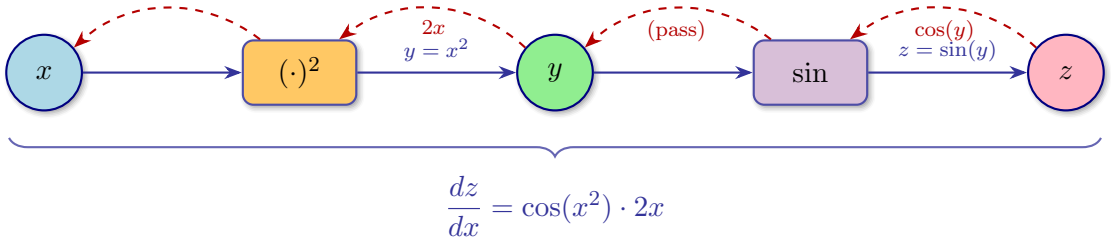

Figure 3: Extended Computation Graph with Chain Rule

4.2.2 Important: Leaf vs Intermediate Tensors

```
# Try to access y's gradient
print(y.grad)
```

Listing 6: Attempting to Access Intermediate Gradients

Error Output:

```
UserWarning: The .grad attribute of a Tensor that is not a leaf Tensor
is being accessed. Its .grad attribute won't be populated during
backward().
None
```

Computation Graph Terminology In PyTorch's computation graph (Directed Acyclic Graph - DAG):

- **Leaf Nodes (Leaves):** Input tensors with `requires_grad=True` (like x)
- **Root Node:** The final output tensor (like z)
- **Intermediate Nodes:** Tensors created during operations (like y)

Default Behavior: Gradients are only computed and stored for **leaf tensors**. Intermediate tensor gradients are not retained to save memory.
Documentation Reference: *"By default, gradients are only retained for leaf tensors. If you need gradients for intermediate tensors, you can use* ***`retain_grad()`****."*

4.3 Example 3: Neural Network Training

Now for the complete neural network example discussed earlier.

```
import torch

# Dataset: Student with CGPA 6.7, no placement (0)
x = torch.tensor(6.7)     # Input: CGPA
y = torch.tensor(0.0)     # Target: No placement

# Binary Cross-Entropy Loss implementation
def binary_cross_entropy_loss(prediction, target):
    epsilon = 1e-8  # Prevent log(0)
    prediction = torch.clamp(prediction, epsilon, 1 - epsilon)
    return -(target * torch.log(prediction) +
             (1 - target) * torch.log(1 - prediction))

# ============================================
# MANUAL BACKPROPAGATION
# ============================================
print("=" * 50)
print("MANUAL BACKPROPAGATION")
print("=" * 50)

w = torch.tensor(1.0)  # Weight
b = torch.tensor(0.0)  # Bias

# Forward pass
```

```
z = w * x + b
y_pred = torch.sigmoid(z)
loss = binary_cross_entropy_loss(y_pred, y)

print(f"Loss: {loss:.4f}")

# Manual derivatives
dloss_dy_pred = (y_pred - y) / (y_pred * (1 - y_pred))
dy_pred_dz = y_pred * (1 - y_pred)
dz_dw = x
dz_db = 1

dL_dw = dloss_dy_pred * dy_pred_dz * dz_dw
dL_db = dloss_dy_pred * dy_pred_dz * dz_db

print(f"Manual dL/dw: {dL_dw:.4f}")
print(f"Manual dL/db: {dL_db:.4f}")

# ============================================
# AUTOGRAD BACKPROPAGATION
# ============================================
print("\n" + "=" * 50)
print("AUTOGRAD BACKPROPAGATION")
print("=" * 50)

# Reset with gradient tracking
w = torch.tensor(1.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

# Forward pass (same as before)
z = w * x + b
y_pred = torch.sigmoid(z)
loss = binary_cross_entropy_loss(y_pred, y)

print(f"Loss: {loss:.4f}")

# Single line for all gradients!
loss.backward()

print(f"Autograd dL/dw: {w.grad:.4f}")
print(f"Autograd dL/db: {b.grad:.4f}")
```

Listing 7: Complete Neural Network with Manual vs Autograd

Output:

```
==================================================
MANUAL BACKPROPAGATION
==================================================
Loss: 6.7012
Manual dL/dw: 6.6918
Manual dL/db: 0.9988

==================================================
AUTOGRAD BACKPROPAGATION
==================================================
Loss: 6.7012
```

```
Autograd dL/dw: 6.6918
Autograd dL/db: 0.9988
```

The Power of Autograd **Manual approach:** Required deriving 4 partial derivatives and implementing chain rule multiplication by hand.
Autograd approach: Just one line - `loss.backward()` - computes all gradients automatically!
Imagine a network with millions of parameters across hundreds of layers. Manual computation is impossible; Autograd makes it trivial.

5 Working with Vector Inputs

So far we've used scalar inputs, but Autograd handles vectors just as easily.

```python
# Create a vector input
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)

# Multi-variable function: y = mean(x^2)
# Mathematically: y = (x1^2 + x2^2 + x3^2) / 3
y = (x ** 2).mean()

print(f"x = {x}")
print(f"y = {y}")

# Backward pass
y.backward()

print(f"\nGradients: {x.grad}")
```

Listing 8: Autograd with Vector Inputs

Output:

```
x = tensor([1., 2., 3.], requires_grad=True)
y = tensor(4.6667, grad_fn=<MeanBackward0>)

Gradients: tensor([0.6667, 1.3333, 2.0000])
```

Mathematical Explanation For vector $\mathbf{x} = [x_1, x_2, x_3]$, the function is:

$$y = \frac{x_1^2 + x_2^2 + x_3^2}{3} \tag{19}$$

Partial derivatives:

$$\frac{\partial y}{\partial x_1} = \frac{2x_1}{3} = \frac{2 \times 1}{3} = 0.6667 \tag{20}$$

$$\frac{\partial y}{\partial x_2} = \frac{2x_2}{3} = \frac{2 \times 2}{3} = 1.3333 \tag{21}$$

$$\frac{\partial y}{\partial x_3} = \frac{2x_3}{3} = \frac{2 \times 3}{3} = 2.0000 \tag{22}$$

Key Insight: When using vector inputs, you're essentially working with **multi-variable functions**. Autograd computes the **gradient vector** (partial derivatives

with respect to each input).

6 Critical Concept: Clearing Gradients

A common and serious mistake in PyTorch is forgetting to clear gradients between iterations.

6.1 The Accumulation Problem

```python
x = torch.tensor(2.0, requires_grad=True)

# First forward-backward pass
y = x ** 2
y.backward()
print(f"First gradient: {x.grad}")  # Output: 4.0 (correct: 2*2=4)

# Second forward-backward pass (WITHOUT clearing)
y = x ** 2
y.backward()
print(f"Second gradient: {x.grad}")  # Output: 8.0 (WRONG! Should be
    4.0)
```

Listing 9: Gradient Accumulation Problem

What happened? The second gradient (4.0) was **added** to the first (4.0), resulting in 8.0!

Danger: Gradient Accumulation By default, PyTorch **accumulates** gradients. This means:

$$\text{x.grad}_{\text{new}} = \text{x.grad}_{\text{old}} + \text{newly computed gradient} \tag{23}$$

This is useful for certain advanced techniques (like gradient accumulation for large batches), but in standard training loops, it causes completely wrong updates!

6.2 The Solution: zero_()

```python
x = torch.tensor(2.0, requires_grad=True)

# First pass
y = x ** 2
y.backward()
print(f"First: {x.grad}")  # 4.0

# Clear gradients before next pass
x.grad.zero_()

# Second pass
y = x ** 2
y.backward()
```

```
print(f"Second: {x.grad}")  # 4.0 (correct!)
```

Listing 10: Proper Gradient Clearing

Best Practice In every training loop, the pattern should be:

1. Forward pass
2. Compute loss
3. `loss.backward()`
4. Update weights
5. `optimizer.zero_grad()` or `tensor.grad.zero_()`

The `zero_()` method (with underscore) modifies the tensor **in-place**, efficiently resetting gradients to zero.

7 Disabling Gradient Tracking

Sometimes you want to perform operations **without** tracking gradients. Common scenarios include:

- **Inference/Prediction:** After training, you only need forward passes
- **Memory efficiency:** Gradient tracking uses extra memory
- **Freezing layers:** When fine-tuning pre-trained models

7.1 Method 1: requires_grad_(False)

Modifies the tensor in-place to stop tracking:

```
x = torch.tensor(2.0, requires_grad=True)
print(f"Before: {x.requires_grad}")  # True

x.requires_grad_(False)
print(f"After: {x.requires_grad}")   # False

y = x ** 2
print(f"y has grad_fn: {y.grad_fn}")  # None

# y.backward()  # ERROR: RuntimeError!
```

Listing 11: Using requires_grad_(False)

7.2 Method 2: detach()

Creates a new tensor sharing the same data but detached from the computation graph:

```
x = torch.tensor(2.0, requires_grad=True)

# Create detached copy
```

```python
z = x.detach()

print(f"x requires_grad: {x.requires_grad}")  # True
print(f"z requires_grad: {z.requires_grad}")  # False
print(f"Same data: {x.eq(z).all()}")          # True

# Operations on original x still track
y = x ** 2
print(f"y grad_fn: {y.grad_fn}")  # <PowBackward0>

# Operations on detached z don't track
y1 = z ** 2
print(f"y1 grad_fn: {y1.grad_fn}")  # None

y.backward()  # Works fine
# y1.backward()  # ERROR!
```

Listing 12: Using detach() to Create New Tensor

7.3 Method 3: torch.no_grad() Context Manager

The most convenient method for temporarily disabling gradients:

```python
x = torch.tensor(2.0, requires_grad=True)

# Normal operation - gradients tracked
y = x ** 2
print(f"With grad tracking: {y.requires_grad}")  # True

# Context manager - gradients disabled
with torch.no_grad():
    y_no_grad = x ** 2
    print(f"Inside no_grad: {y_no_grad.requires_grad}")  # False

# Outside context, tracking resumes
y_normal = x ** 2
print(f"Outside no_grad: {y_normal.requires_grad}")  # True
```

Listing 13: Using no_grad() Context Manager

When to Use Each Method

- `requires_grad_(False)`: Permanently disable tracking for a parameter (e.g., freezing layers)
- `detach()`: Create a copy for use in other computations without affecting the original graph
- `torch.no_grad()`: Temporary disabling for inference or evaluation (most common)

8 Complete Training Loop Example

Putting it all together, here's how Autograd fits into a proper training workflow:

```python
import torch
import torch.nn as nn
import torch.optim as optim

# Generate synthetic data: y = 2x + 1 + noise
torch.manual_seed(42)
X = torch.randn(100, 1)
y_true = 2 * X + 1 + 0.1 * torch.randn(100, 1)

# Initialize parameters with gradient tracking
w = torch.randn(1, requires_grad=True)
b = torch.randn(1, requires_grad=True)

# Hyperparameters
learning_rate = 0.1
epochs = 100

# Optimizer (handles parameter updates)
optimizer = optim.SGD([w, b], lr=learning_rate)

print(f"Initial: w={w.item():.4f}, b={b.item():.4f}")

for epoch in range(epochs):
    # ========== FORWARD PASS ==========
    y_pred = w * X + b
    loss = ((y_pred - y_true) ** 2).mean()  # MSE

    # ========== BACKWARD PASS ==========
    optimizer.zero_grad()  # Clear previous gradients
    loss.backward()        # Compute all gradients

    # ========== UPDATE PARAMETERS ==========
    optimizer.step()     # w = w - lr * w.grad

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1}: Loss={loss.item():.4f}, "
              f"w={w.item():.4f}, b={b.item():.4f}")

print(f"\nFinal: w={w.item():.4f} (target: 2.0), "
      f"b={b.item():.4f} (target: 1.0)")
```

Listing 14: Complete Training Loop with Autograd

Key Observations:

1. `requires_grad=True` is set when creating parameters
2. `optimizer.zero_grad()` clears gradients each iteration
3. `loss.backward()` computes $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$ automatically
4. `optimizer.step()` updates parameters using computed gradients

9 Summary and Key Takeaways

Autograd Essentials

1. **Purpose:** Automatic computation of gradients for neural network training
2. **Mechanism:** Builds dynamic computation graph during forward pass
3. **Usage:** Set `requires_grad=True` on input tensors
4. **Execution:** Call `.backward()` on scalar output to compute all gradients
5. **Access:** Retrieve gradients via `.grad` attribute
6. **Clearing:** Always use `.zero_grad()` before new backward passes
7. **Disabling:** Use `torch.no_grad()` for inference to save memory

9.1 Why Autograd is Indispensable

- **Neural networks are nested functions:** Each layer feeds into the next
- **Chain rule is mandatory:** Gradients flow backward through all layers
- **Manual computation is impossible:** Modern networks have millions of parameters
- **Autograd is automatic:** One line (`.backward()`) handles everything

"You cannot train neural networks in PyTorch without Autograd. It is a very, very integral part of PyTorch."

10 Quick Reference Card

Operation	Code	Purpose
Enable tracking	`x = torch.tensor(3.0, requires_grad=True)`	Track operations on x
Check tracking	`x.requires_grad`	Returns True/False
Compute gradients	`y.backward()`	Compute $\partial y/\partial x$ for all x
Access gradient	`x.grad`	Get computed gradient value
Clear gradients	`x.grad.zero_()`	Reset to zero (in-place)
Disable (permanent)	`x.requires_grad_(False)`	Stop tracking permanently
Detach tensor	`z = x.detach()`	New tensor, no tracking
Disable (temp)	`with torch.no_grad():`	Context manager for inference

Table 1: Autograd Quick Reference